

| 第十周作业 -1

| 第十周作业 -1

| 基础作业

| 1.1 运行课堂 Demo，描述处理流程

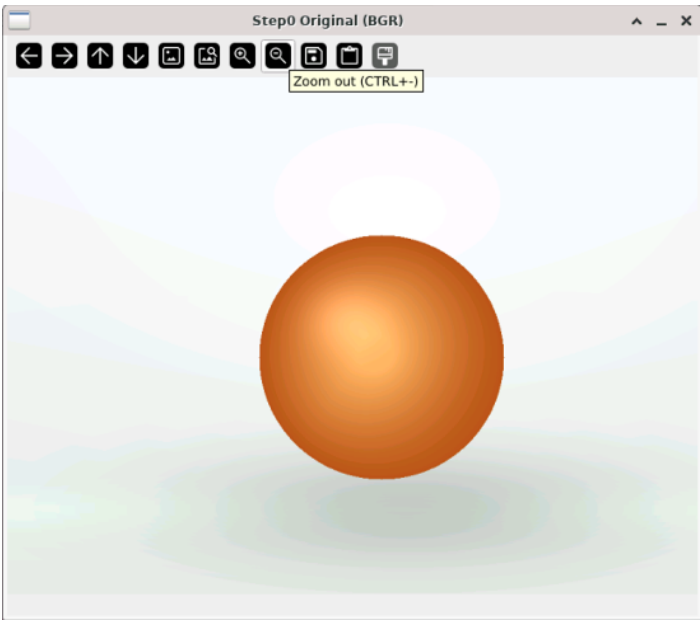
运行课程提供的 `demo.py`（任选一张图），观察每一步弹出的窗口，用自己的话描述以下每个步骤分别做了什么、输出的图像长什么样：

- 1. Step 0: 读图
- 2. Step 1: RGB 通道拆分
- 3. Step 2: 转 HSV 并拆通道
- 4. Step 3: 高斯滤波
- 5. Step 4: HSV 阈值分割（mask）
- 6. Step 5: 形态学处理
- 7. Step 6: 轮廓提取与质心输出

每步用 2-3 句话描述即可，并截图保留各步的输出窗口（或 `outputs/` 中对应的保存图）。

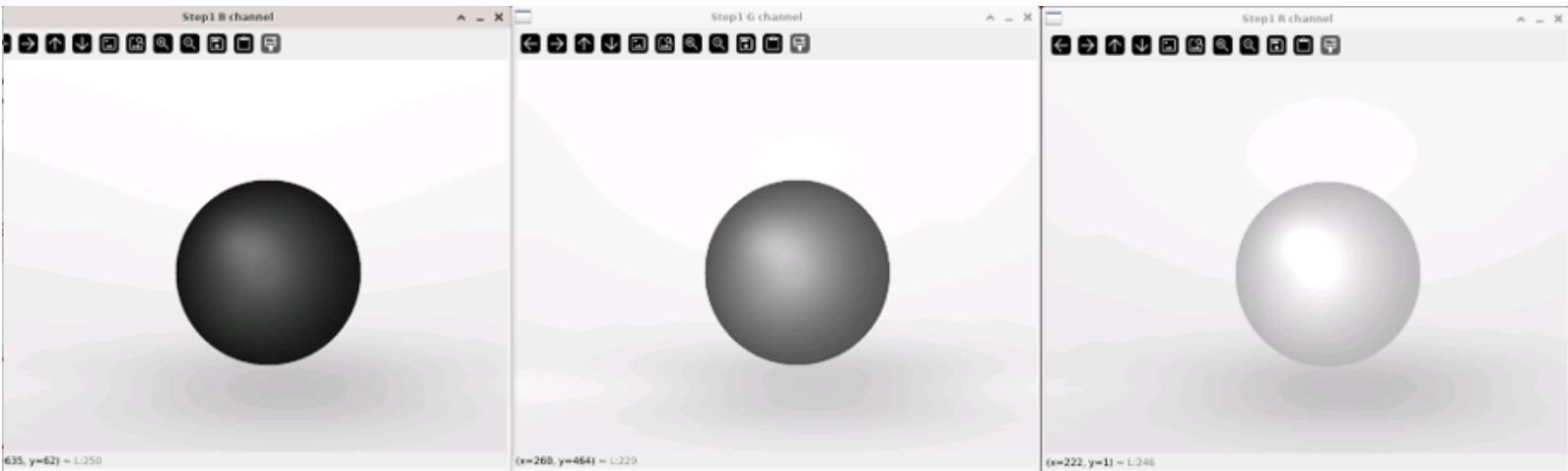
1. Step 0: 读图 (`step0_read_and_show`)

- 做了什么：使用 `cv2.imread` 读取输入图片，默认读取出的色彩空间是 BGR 格式。随后控制台会打印出图像的尺寸（高、宽、通道数）和数据类型。如果在屏幕上显示过大，内部函数 `_fit` 会等比缩放图像。
- 输出的图像长什么样：弹出一个窗口显示原图（普通的彩色图片，画面中包含待检测的橙色小球及背景）。



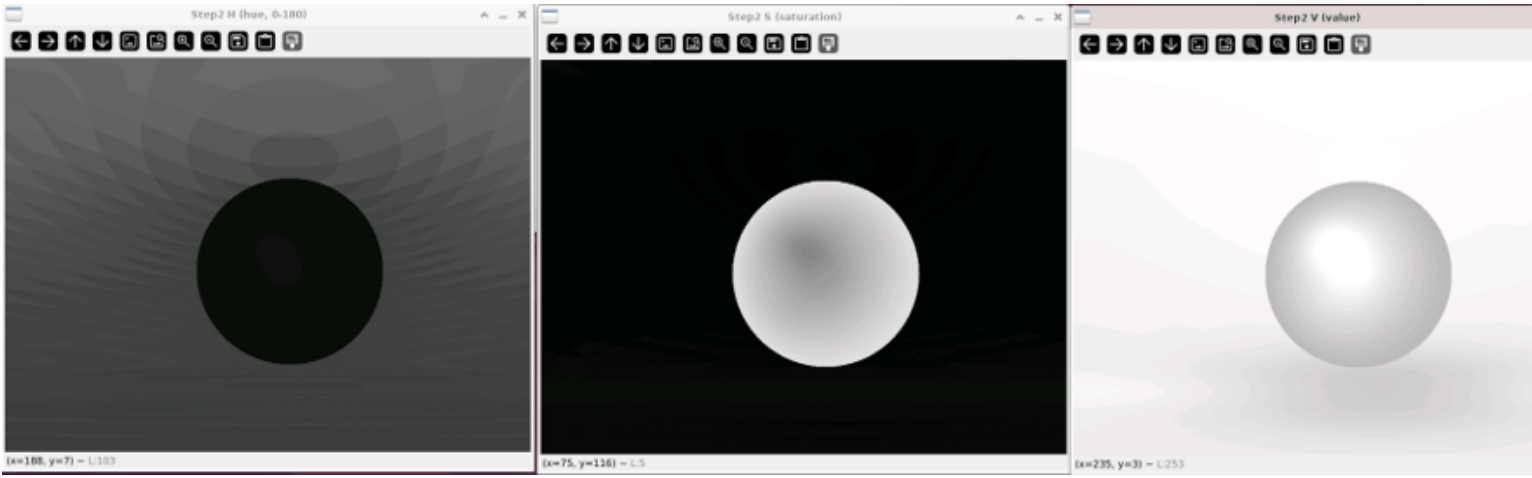
2. Step 1: RBG 通道拆分 (`step1_rgb_channels`)

- 做了什么：使用 `cv2.split(img)` 将原本的 BGR 图像拆分成 Blue、Green、Red 三个独立的单通道图像，并打印这三个通道的平均亮度。
- 输出的图像长什么样：分别弹出三个窗口显示 B、G、R 通道的图像，全都是灰度图（黑白画面）。由于目标是橙色球，看到在 R（红色）通道图像中，球的区域非常亮（偏白）；而在 B（蓝色）通道图像中，球的区域非常暗（偏黑）。



3. Step 2: 转 HSV 并拆通道 (`step2_hsv_channels`)

- 做了什么：通过 `cv2.cvtColor` 将原图从 BGR 转换到 HSV（色调、饱和度、明度）色彩空间。接着同样使用 `cv2.split` 拆分成 H、S、V 三个单通道。
- 输出的图像长什么样：弹出三个窗口，分别为 H、S、V 三个通道的灰度图。其中最重要的是 H（色调）通道图像，在这张图里，橙色球因为颜色纯粹，会呈现出一块灰度值非常均匀且稳定的区域，这就为后续的阈值分割提供了依据。



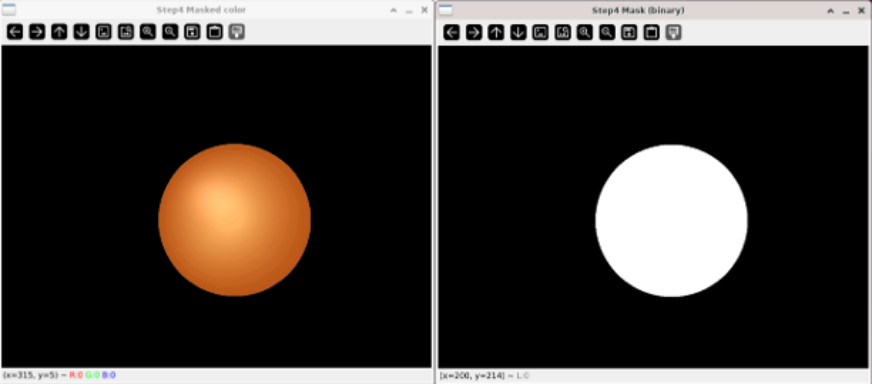
4. Step 3: 高斯滤波 (`step3_blur`)

- 做了什么：使用 `cv2.GaussianBlur`（5x5 的卷积核）对图像进行平滑处理，目的是消除图像中的高频噪声（如细小颗粒和噪点），让颜色区域更加平滑稳定。
- 输出的图像长什么样：弹出一个窗口，左右拼接了“滤波前”和“滤波后”的彩色对比图。右侧的高斯滤波图像看起来会稍微有些模糊和柔和，细微的噪点被抹平了。



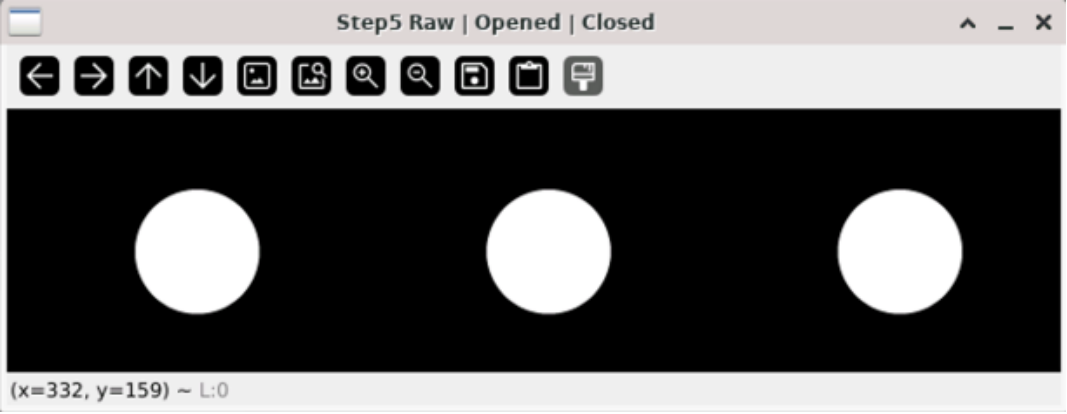
5. Step 4: HSV 阈值分割 (mask) (`step4_hsv_mask`)

- 做了什么：先将滤波后的图像转为 HSV，然后使用 `cv2.inRange` 和预设的上下限阈值（提取橙色的范围）生成一个二值化的 Mask（掩膜）。最后用 `cv2.bitwise_and` 将 Mask 盖在原图上，把小球“抠”出来。
- 输出的图像长什么样：弹出两个窗口。
 - Mask (binary)：一张黑白二值图，其中匹配橙色的区域（球）是纯白色的，背景全是纯黑色的。
 - Masked color：一张彩色图，背景全黑，只有小球的区域保留了原本的彩色。



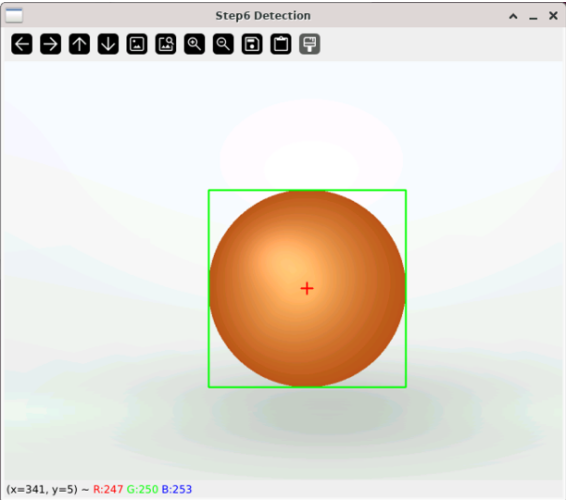
6. Step 5: 形态学处理 (`step5_morphology`)

- 做了什么：利用 9x9 的椭圆结构核，对上一步得到的 Mask 进行形态学操作。
- 输出的图像长什么样：弹出一个窗口，将“原始 Mask”、“开运算后”、“闭运算后”三张黑白二值图并排拼接在一起。看到图像从带有瑕疵的白斑，变成了一个边缘平滑、内部饱满且周围干净的纯白色圆形区域。



7. Step 6: 轮廓提取与质心输出 (`step6_contour_centroid`)

- 做了什么：在纯净的 Mask 图上使用 `cv2.findContours` 寻找仅最外层的轮廓。筛选出面积最大的轮廓后，用 `cv2.boundingRect` 计算出它的外接矩形（包围盒），并利用 `cv2.moments` (图像矩) 计算出该轮廓的几何质心（即小球的中心点 x, y 坐标）。
- 输出的图像长什么样：弹出一个窗口显示最终的检测结果图。这是一张彩色原图，但是小球的外面被画上了一个绿色的矩形方框，并且小球的正中心被画上了一个红色的十字准星（表示算出来的质心位置）。



1.2 代码结构分析

阅读 `demo.py`，回答以下问题：

1. `cv2.split(img)` 返回的三个通道顺序是什么？
2. `cv2.inRange(hsv, HSV_LOWER, HSV_UPPER)` 的返回值是什么数据类型？其中 "前景像素" 的值是多少、"背景像素" 的值是多少？
3. `cv2.moments(c)` 返回的 `m["m00"]` 物理上代表什么？
 - 质心公式 $c_x = M_{10} / M_{00}$ ，其中 M_{10} 的含义是什么？
 - 代码中为什么要先检查 `if m["m00"]`，不检查会有什么风险
4. `step5_morphology` 中，先做开运算 (open) 再做闭运算 (close)，如果顺序反过来，先 close 再 open，会有什么不同效果？可以自行选取一些足球照片并结合 mask 图说明。

1. OpenCV 中默认读取和存储图像的色彩空间顺序是 BGR。因此，`cv2.split(img)` 返回的三个通道顺序依次是：B（Blue/蓝色）、G（Green/绿色）、R（Red/红色）。
2. 返回值数据类型：返回的是一个单通道的二维数组（`np.ndarray`），数据类型默认为 `uint8`。物理意义上它是一个二值图像（掩膜 Mask）。前景像素的值：如果原图像素的值在 `[HSV_LOWER, HSV_UPPER]` 范围内，则判定为前景，其像素值会被设为 `255`（在灰度图上表现为纯白色）。

背景像素的值：如果不在这个范围内，则判定为背景，像素值会被设为 0（在灰度图上表现为纯黑色）。

3. `m["m00"]` 物理上代表什么？ `m["m00"]` 也就是图像的零阶矩（0th order moment）。对于二值图像或轮廓而言，它物理上代表该轮廓区域的面积（即区域内非零像素的总和）。
- 为什么要先检查 `if m["m00"]`，不检查会有什么风险？ 如果不做检查，当提取到的轮廓极其微小（比如单像素噪点或一条直线），其包围的面积可能被计算为 `0.0`。如果此时直接执行 `$M_{10} / M_{00}$`，就会触发除以零错误（`ZeroDivisionError`）导致程序崩溃。加上判断后，当分母为 0 时可以安全退回到使用外接矩形中心的备用方案（`x + w // 2`）。

4. 形态学处理（Open 与 Close）的顺序影响：

在目标检测中，形态学运算的顺序非常关键：

- 开运算（**Open** = 先腐蚀后膨胀）：主要用于去除背景中的细小噪点、断开两块黏连的物体。
 - 闭运算（**Close** = 先膨胀后腐蚀）：主要用于填补目标物体内部的小型空洞、连接断裂的部分。
- 如果顺序反过来（先 **Close** 再 **Open**）会有什么不同效果？
- 原顺序（先开后闭，当前代码）： 首先把背景里散落的孤立噪点直接“抹除”掉；然后再把主角（小球）内部可能存在的黑洞给“填满”。得到的结果非常干净。
 - 反过来顺序（先闭后开）： 闭运算的第一步是“膨胀”。如果背景中原本有一些散落、但距离比较近的细碎噪点，一经膨胀，它们就连成了一大片噪声块。随后的腐蚀由于噪声块已经变大，无法将其消除。这会导致背景中残留巨大的伪目标斑块。
- 结合场景说明： 假设我们在草地上识别一个橘黄色的足球：
- 由于草地反光或落叶干扰，Mask 背景上可能会有十几个散碎的小白点（噪点）；同时由于足球表面的花纹或脏污，足球这个大白斑中间可能空了几个小黑洞。
 - 如果先闭后开：闭运算会先把那些靠得很近的草地反光小白点连在一起，变成一块不小的白斑，顺带填上了足球上的黑洞。紧接着的开运算已经无法腐蚀掉这块新形成的草地白斑了。最后你会检测到“两个”甚至更多目标。
 - 如果先开后闭（正确做法）：开运算直接把草地上的零星小白点“掐死”在摇篮里，足球虽然小黑洞变大了些，但在随后的闭运算中，足球的黑洞被重新补满。最后只会得到一个完美干净的足球。

1.3 颜色空间与滤波方法拓展

1. 课堂只用了 RGB 和 HSV 两种颜色空间。请查阅资料，再列出至少 3 种颜色空间，并对每种简述：
- 三个通道各代表什么含义
 - 与 RGB 和 HSV 相比，它在什么场景下更有优势
2. 课堂只讲解了高斯滤波和中值滤波。请查阅资料，再列出至少 3 种滤波方法，并说明：
- 这些滤波方法的核心思想是什么
 - 它适合处理什么类型的噪声或图像

其他颜色空间说明

1. LAB (CIELAB) 颜色空间

- 三个通道代表的含义：
 - L (Lightness)**：明度分量，表示亮度（从黑到白，0~100）。
 - a**：洋红/绿轴色度分量（负值为绿色，正值为洋红色/红色）。
 - b**：黄/蓝轴色度分量（负值为蓝色，正值为黄色）。
- 应用优势：
 - 感知均匀性（**Perceptually Uniform**）：它是最接近人类视觉生理特性的颜色空间。在 LAB 空间中，两种颜色在数值上的欧氏距离，几乎完美对应了人类肉眼感知到的“色差”。
 - 相比 **RGB/HSV** 的优势场景：在需要精准计算色差、颜色匹配、以及图像色彩校正/调色的场景下具有绝对优势（如工业品色差检测、Photoshop 中的高级调色）。因为在 RGB 和 HSV 中，数值改变 10 并不意味着人眼看起来颜色变化程度是一样的，而 LAB 做到了这一点。

2. YCrCb (或称 YCbCr) 颜色空间

- 三个通道代表的含义：
 - Y (Luma/Luminance)**：亮度分量，包含了图像的所有灰度信息。
 - Cr (Chroma Red)**：红色色差分量，反映了 RGB 输入信号红色部分与 RGB 亮度值之间的差异。
 - Cb (Chroma Blue)**：蓝色色差分量，反映了 RGB 输入信号蓝色部分与 RGB 亮度值之间的差异。
- 应用优势：
 - 亮色彻底分离：人类眼睛对亮度极度敏感，而对色彩细节（色度）相对不敏感。YCrCb 将亮度 (Y) 和颜色 (Cr, Cb) 进行了彻底分离。
 - 相比 **RGB/HSV** 的优势场景：
 - 图像/视频压缩：它是 JPEG 和 MPEG（如 H.264）视频压缩的标准空间。因为可以将 Cr 和 Cb 通道进行下采样（降低分辨率）而人眼几乎察觉不到画质下降，从而大幅减小文件体积，这是 RGB 无法做到的。
 - 肤色检测与人脸识别：研究表明，无论是黑种人、白种人还是黄种人，在 YCrCb 空间去除 Y 通道后，其 Cr 和 Cb 的分布非常集中，非常适合用于排除光照干扰的肤色提取。

3. HSL 颜色空间

- 三个通道代表的含义：
 - H (Hue)**：色相，与 HSV 相同，表示颜色的种类（如红、黄、蓝），取值角度为 0° 到 360°。
 - S (Saturation)**：饱和度，表示色彩的纯度或鲜艳程度。
 - L (Lightness)**：亮度（或称明度），表示颜色的明暗程度。
- 应用优势（与 HSV 的细微差别）：
 - 更符合直觉的“白色”定义：在 HSV 中，V（Value）达到最大值 100% 时，它代表的是“颜色最纯/最亮”，例如 V=100%, S=100% 对应的是极其鲜艳的纯红色。而在 HSL 中，无论什么颜色，只要 L（Lightness）达到 100%，结果就是纯白色；L 为 0% 就是纯黑色。这更符合人类在现实生活中“加白漆”或“加黑漆”的直觉。
 - 相比 **RGB/HSV** 的优势场景：HSL 在前端网页设计（**CSS3 支持 `hsl()`**）、**UI/UX** 界面设计中更具优势。设计师想要同一个主题色在悬停时变亮、按下时变暗，只需简单调节 HSL 中的 L 分量即可，这在 RGB 中很难计算，在 HSV 中有时也不如 HSL 直观。

其他滤波方法说明

1. 均值滤波 (Mean Filtering)

- **核心思想：**
这是最基础的线性滤波方法。它将滤波器（卷积核）覆盖区域内所有像素的值进行简单相加，然后求平均值，用这个平均值来替换中心像素的值。
- **适合处理的噪声/图像：**
 - 适合：处理具有均匀分布规律的噪声，或者用于需要快速大面积模糊图像（如生成阴影、降低图像分辨率前处理）的场景。
 - 缺点：非常“暴力”，在平滑噪声的同时会严重破坏图像的边缘细节，导致图像整体变得十分模糊。

2. 双边滤波 (Bilateral Filtering)

- **核心思想：**
双边滤波是一种非线性滤波方法，它的核心在于“保边去噪”。它在计算平均像素值时，不仅考虑了空间距离（距离中心越近的像素权重越大，即高斯空间权重），还引入了像素值差异的考量（颜色/灰度差值越小的像素权重越大，即高斯像素范围权重）。这意味着如果遇到图像边缘（像素值剧烈变化的地方），边缘另一侧的像素由于颜色差异大，其权重会趋近于 0，从而不会参与平均。
- **适合处理的噪声/图像：**
 - 适合：需要去除噪声但必须保留清晰锐利边缘的图像。
 - 典型应用：美颜相机中的“磨皮”功能（滤除皮肤上的斑点和纹理，但保留眼睛、嘴巴等五官的边缘）；也常用于产生卡通化、水彩画效果的图像前处理。

3. 非局部均值滤波 (Non-Local Means Filtering, NLM)

- **核心思想：**
与均值、高斯等“只看周围一小圈邻居”的局部滤波不同，NLM 会在整幅图像（或一个非常大的搜索窗口内）寻找与当前像素周围“结构/纹理相似”的图像块。它将这些相似图像块的中心像素赋予更高的权重，然后进行加权平均。简而言之，它是利用图像本身存在的自相似性（如重复的砖墙纹理、草地）来进行去噪。
- **适合处理的噪声/图像：**
 - 适合：处理高斯白噪声（AWGN），尤其是包含大量重复纹理、细小细节的高质量图像。
 - 优势：去噪效果极其惊艳，既能抹除明显的噪点，又能完美保留衣服纹理、毛发等微观细节，是传统去噪算法中画质最好的一种。
 - 缺点：计算量极大，处理速度较慢，通常不适用于对实时性要求极高的场景。

1.4 Aelos 视觉回传基本使用

1. 使用 Aelos 机器人进行视觉识别，至少能够识别一种颜色；
2. 分析 Aelos 自带的视觉回传功能是如何实现的，与课程讲解的图像处理流程相比，有哪些异同；
3. 让机器人在识别到指定颜色后执行一个自定义动作，提交识别流程截图与动作执行效果截图。

1.5 SMART 基础功能探索

1. 阅读并完成 SMART 基础使用教程；
2. 在教程基础上，自行实现至少一个额外功能（如传感器读取、设计一个自动化流程等，类型不限）；
3. 简要说明你为什么选择这个功能、实现过程中遇到了什么问题，提交使用流程说明及效果照片或截图。

基础作业提交要求

- 整理为 一份 PDF，包含：
- 1.1 各步截图（每步一张，共 7 张）及文字描述
 - 1.2 四道代码分析题的回答
 - 1.3 两道拓展知识题的回答
 - 1.4 Aelos 视觉识别调试截图、动作执行效果截图，及与课程流程的对比分析
 - 1.5 额外功能说明及效果截图